

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-clear! greetings)
  (ht-items greetings))
```

```
nil
```

An entire hash table can be copied to another hash table using the *ht-copy* API as shown below:

```
(ht-copy hash-table) ;; Syntax
```

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-items (ht-copy greetings)))
```

```
(( "Adam" "Hello Adam!" ))
```

The *ht-merge* function can combine two different hash tables into one. In the following example, the items in the *english* and *numbers* hash tables are merged together.

```
(ht-merge hash-table1 hash-table2) ;; Syntax
```

```
(let ((english (ht-create))
      (numbers (ht-create)))
  (ht-set! english "a" "A")
  (ht-set! numbers "1" "One")
  (ht-items (ht-merge english numbers)))
```

```
(( "1" "One" ) ( "a" "A" ))
```

You can make modifications to an existing hash table. For example, you can remove an item in the hash table using the *ht-remove!* function, which takes as input a hash table and a key as shown below:

```
(ht-remove hash-table key) ;; Syntax
```

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-set! greetings "Eve" "Hello Eve!")
  (ht-remove! greetings "Eve")
  (ht-items greetings))
```

```
(( "Adam" "Hello Adam!" ))
```

You can do an in-place modification to an item in the hash table using the *ht-update!* function. An example is given below:

```
(ht-update! hash-table key value) ;; Syntax
```

```
(let ((greetings (ht-create)))
```

```
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-update! greetings (ht ("Adam" "Howdy Adam!")))
  (ht-items greetings))
```

```
(( "Adam" "Howdy Adam!" ))
```

A number of predicate functions are available in *ht.el* that can be used to check for conditions in a hash table. The *ht?* function checks to see if the input argument is a hash table. It returns *t* if the argument is a hash table and *nil* otherwise.

```
(ht? hash-table) ;; Syntax
```

```
(ht? nil)
```

```
nil
```

```
(let ((greetings (ht-create)))
  (ht? greetings))
```

```
t
```

You can verify if a key is present in a hash table using the *ht-contains?* API, which takes a hash table and key as arguments. It returns *t* if the item exists in the hash table. Otherwise, it simply returns *nil*.

```
(ht-contains? hash-table key) ;; Syntax
```

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-contains? greetings "Adam"))
```

```
t
```

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-contains? greetings "Eve"))
```

```
nil
```

The *ht-empty?* function can be used to check if the input hash-table is empty or not. A couple of examples are shown below:

```
(ht-empty? hash-table) ;; Syntax
```

```
(let ((greetings (ht-create)))
  (ht-set! greetings "Adam" "Hello Adam!")
  (ht-empty? greetings))
```

```
nil
```